

Cloud & DevOps Project Report

Name: Vikhyat Saini

Reg No.: 12322613

Section: 4OM56

Roll No.: 49

Faculty Name: Ms. Arshiya

GitHub Link: <https://github.com/VikhyatSaini/VirtualStylist>



Abstract

The rapid growth of cloud computing and DevOps practices has transformed the software development lifecycle by enabling faster delivery, improved scalability, continuous integration, and automated deployment. This project focuses on deploying a Virtual Stylist web application using modern DevOps tools and cloud technologies.

The project integrates multiple technologies including Docker for containerization, Jenkins for Continuous Integration and Continuous Deployment (CI/CD), Kubernetes (K3s) for orchestration, Terraform for Infrastructure as Code (IaC), AWS EC2 for cloud hosting, Prometheus for monitoring, Grafana for visualization, and GitHub for version control.

The Virtual Stylist application is developed using Flask and provides users with outfit suggestions and fashion-related interactions through a web-based interface. The application is containerized using Docker and automatically built and deployed using Jenkins pipelines. Kubernetes manages application deployment and scaling while Prometheus and Grafana monitor the overall system performance.

The project demonstrates the implementation of a complete DevOps lifecycle, reducing manual deployment efforts while improving reliability, scalability, automation, and monitoring capabilities.

Table of Contents

Introduction
Problem Statement
Objectives
Scope of the Project
Literature Review
Existing System
Proposed System
System Architecture
Technologies Used
Methodology
Implementation
Terraform Configuration
Docker Containerization
Jenkins CI/CD Pipeline
Kubernetes Deployment
Monitoring using Prometheus and Grafana
Testing and Validation
Advantages of the System
Challenges Faced
Future Enhancements
Conclusion
References

Introduction

Modern software development requires automation, scalability, rapid deployment, and continuous monitoring. Traditional deployment methods involve manual configuration and deployment processes which are time-consuming, error-prone, and difficult to scale.

DevOps has emerged as a methodology that combines software development and IT operations to automate software delivery and infrastructure management. Cloud computing further enhances DevOps by providing scalable and on-demand resources.

This project demonstrates the deployment of a Virtual Stylist web application using a complete DevOps pipeline on AWS cloud infrastructure.

The Virtual Stylist application assists users with fashion recommendations and outfit suggestions through a user-friendly interface. The deployment process integrates various DevOps tools to automate build, deployment, monitoring, and management processes.

The project aims to provide practical exposure to modern software deployment techniques and cloud-native application management.

Problem Statement

Traditional application deployment methods suffer from several limitations such as:

- Manual deployment processes
- High deployment time
- Lack of scalability
- Difficult rollback mechanisms
- Inconsistent environments
- Limited monitoring capabilities
- Increased chances of human error

These issues reduce development efficiency and make application management difficult.

Therefore, there is a need for an automated deployment system that can:

- Automate application builds and deployments
- Ensure consistency across environments
- Improve scalability and reliability
- Enable continuous integration and continuous deployment
- Provide real-time monitoring and visualization

Objectives

The main objectives of this project are:

1. To develop and deploy a Virtual Stylist web application.
2. To implement Infrastructure as Code using Terraform.
3. To containerize the application using Docker.
4. To automate build and deployment processes using Jenkins.
5. To deploy the application on Kubernetes.
6. To host the infrastructure on AWS EC2.
7. To implement monitoring using Prometheus and Grafana.
8. To demonstrate a complete DevOps lifecycle implementation.

Scope of the Project

The project focuses on implementing cloud-native deployment practices for a web-based application.

Scope includes:

- Cloud infrastructure provisioning
- CI/CD pipeline creation
- Container orchestration
- Automated deployment
- Monitoring and visualization
- Cloud hosting and management

The project does not focus on advanced AI-based fashion recommendation algorithms but primarily emphasizes DevOps implementation and deployment automation.

Literature Review

DevOps practices have gained popularity due to their ability to bridge the gap between development and operations teams.

Docker

Docker is a containerization platform that packages applications along with dependencies into lightweight containers.

Kubernetes

Kubernetes is an orchestration platform used for managing containerized applications at scale.

Jenkins

Jenkins is an open-source automation server used for Continuous Integration and Continuous Deployment.

Terraform

Terraform is an Infrastructure as Code tool used to provision and manage cloud infrastructure.

Prometheus and Grafana

Prometheus is used for metrics collection while Grafana provides visualization dashboards.

The combination of these technologies creates a modern DevOps ecosystem for efficient application deployment.

Existing System

Traditional deployment approaches generally involve:

- Manual server setup
- Manual dependency installation
- Direct deployment on servers
- Lack of automation
- Limited monitoring

Disadvantages of existing systems:

- Time-consuming deployment
- Difficult scalability
- Increased downtime
- Lack of consistency

- Poor resource utilization

Proposed System

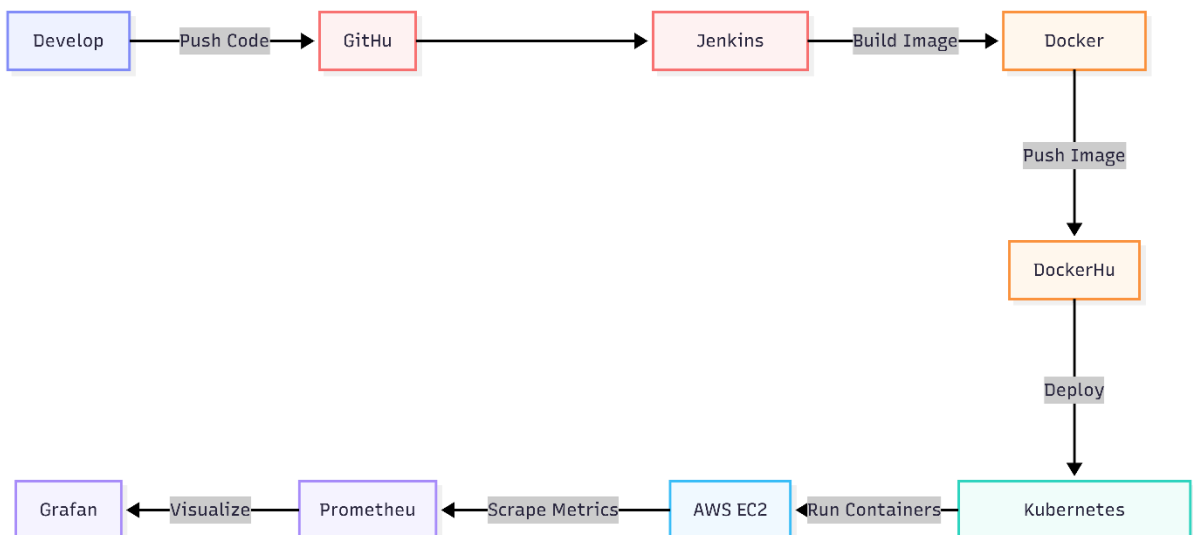
The proposed system automates the complete deployment lifecycle using DevOps practices.

Features of proposed system:

- Automated CI/CD pipeline
- Containerized deployment
- Kubernetes orchestration
- Cloud infrastructure provisioning
- Real-time monitoring
- Improved scalability and reliability

The proposed architecture integrates GitHub, Jenkins, Docker, Kubernetes, AWS, Prometheus, and Grafana.

System Architecture



Components

GitHub

Stores source code and version history.

Jenkins

Automates CI/CD pipeline execution.

Docker

Creates isolated application containers.

DockerHub

Stores Docker images.

Kubernetes

Manages container deployment and orchestration.

AWS EC2

Provides cloud infrastructure.

Prometheus

Collects monitoring metrics.

Grafana

Visualizes monitoring data.

Technologies Used

Technology	Purpose
Python Flask	Web Application Development
HTML/CSS/JavaScript	Frontend Interface
Docker	Containerization
Jenkins	CI/CD Automation
Kubernetes (K3s)	Container Orchestration
Terraform	Infrastructure Provisioning
AWS EC2	Cloud Hosting
GitHub	Version Control

Prometheus	Monitoring
Grafana	Visualization
DockerHub	Image Repository

Methodology

The project follows a DevOps-oriented software deployment methodology where application development, infrastructure provisioning, containerization, deployment automation, orchestration, and monitoring are integrated into a unified workflow.

The methodology adopted in this project ensures faster deployment, reduced manual intervention, improved scalability, and continuous monitoring.

Development Lifecycle

The complete workflow of the project is divided into multiple phases:

1. Application Development
2. Version Control Management
3. Infrastructure Provisioning
4. Containerization
5. CI/CD Pipeline Automation
6. Kubernetes Deployment
7. Monitoring and Visualization

Step 1: Application Development

The Virtual Stylist application was developed using the Flask framework in Python. Flask was selected because of its lightweight architecture, simplicity, flexibility, and compatibility with containerized deployments.

The frontend of the application was designed using:

- HTML
- CSS

- JavaScript

The application provides a user-friendly interface where users can interact with the virtual stylist and obtain fashion-related suggestions.

Step 2: Version Control using GitHub

The source code was uploaded to GitHub for centralized version control and collaboration.

GitHub provided the following benefits:

- Code backup
- Version tracking
- Collaboration support
- Integration with Jenkins CI/CD pipeline
- Easy rollback to previous versions

The repository contains:

- Source code
- Docker configuration files
- Jenkins pipeline configuration
- Kubernetes manifests
- Terraform scripts
- Monitoring configurations

Step 3: Infrastructure Provisioning using Terraform

Terraform was used to automate the provisioning of AWS cloud infrastructure.

The infrastructure created included:

- AWS EC2 instance
- Security groups
- Network access configuration

Terraform simplified infrastructure management by enabling Infrastructure as Code (IaC), where infrastructure is defined through configuration files instead of manual setup.

Terraform Workflow

terraform init

terraform plan

terraform apply

Advantages of Terraform

- Reusable infrastructure configuration
- Faster provisioning
- Reduced human error
- Easy infrastructure modification
- Version-controlled infrastructure

Step 4: Docker Containerization

Docker was used to package the application along with all dependencies into lightweight and portable containers.

The Docker container ensured:

- Environment consistency
- Dependency isolation
- Faster deployment
- Platform independence

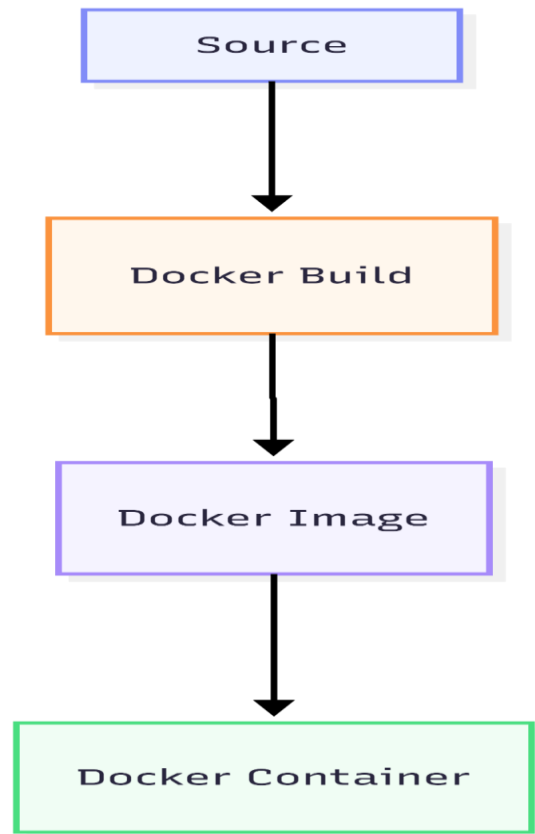
Docker Build Process

The Dockerfile defined:

- Base image
- Dependency installation
- Application copy commands
- Port exposure

- Startup command

Docker Workflow



Step 5: Jenkins CI/CD Pipeline

Jenkins was configured to automate the Continuous Integration and Continuous Deployment process.

The Jenkins pipeline performs the following operations:

1. Pull latest source code from GitHub
2. Build Docker image
3. Push Docker image to DockerHub

This automation reduced manual deployment effort and ensured consistent deployment processes.

Jenkins Pipeline Stages

Stage 1: Checkout SCM

Pulls latest application source code from GitHub repository.

Stage 2: Build Docker Image

Builds Docker image using Dockerfile.

Stage 3: Push Docker Image

Pushes generated Docker image to DockerHub repository.

Step 6: Kubernetes Deployment

Kubernetes (K3s) was used to orchestrate and manage containerized application deployment.

The application was deployed using:

- Deployment YAML
- Service YAML

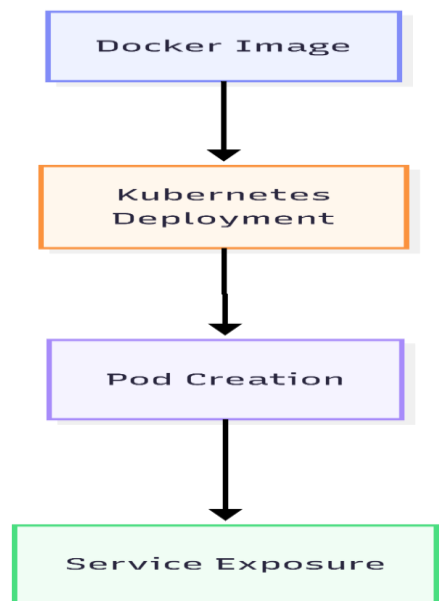
Deployment Responsibilities

- Pod management
- Replica management
- Automatic recovery
- Rolling updates

Service Responsibilities

- Exposing application to external users
- Load balancing traffic
- NodePort mapping

Kubernetes Workflow



Step 7: Monitoring using Prometheus and Grafana

A monitoring stack was implemented using:

- Node Exporter
- Prometheus
- Grafana

Node Exporter

Collects system-level metrics from the EC2 instance.

Prometheus

Scrapes and stores monitoring metrics.

Grafana

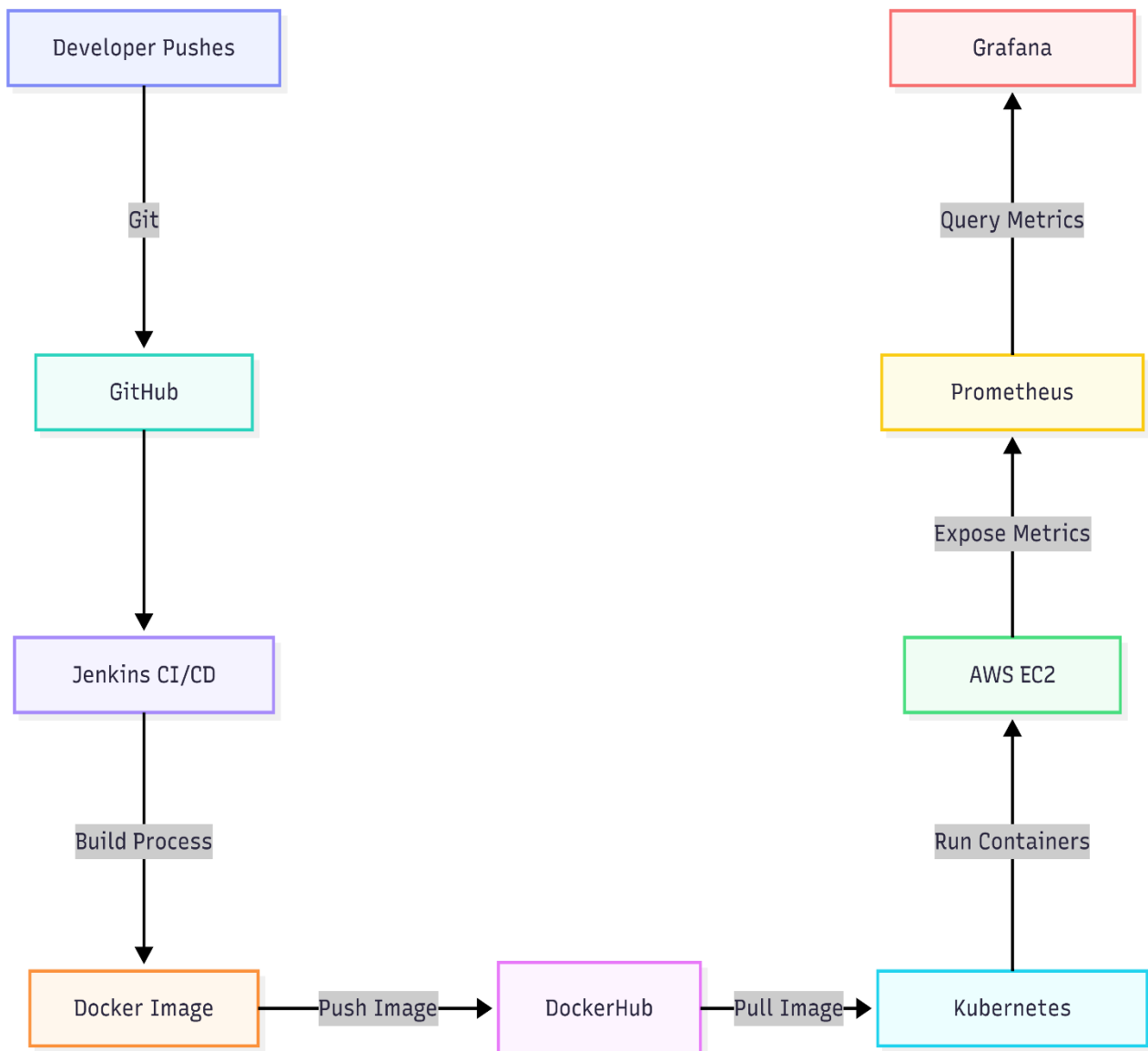
Visualizes metrics using dashboards and graphs.

Metrics Monitored

- CPU utilization
- Memory usage

- Disk usage
- Network traffic
- System uptime

Overall Workflow of the System



This methodology demonstrates the practical implementation of a complete DevOps lifecycle for modern cloud-native applications.

Implementation

Overview

The implementation phase involved integrating multiple DevOps tools and cloud technologies to automate deployment, monitoring, and management of the Virtual Stylist application.

The implementation was divided into several stages including application development, cloud infrastructure provisioning, containerization, CI/CD setup, Kubernetes deployment, and monitoring integration.

Flask Application Development

The Virtual Stylist application was developed using the Flask framework.

Features of the Application

- Interactive user interface
- Fashion recommendation system
- Outfit suggestion interface
- Responsive design
- User login and interaction pages

Backend Implementation

The backend was developed in Python using Flask.

Main responsibilities of backend:

- Handling routes
- Processing user requests
- Rendering templates
- Managing application logic

Frontend Implementation

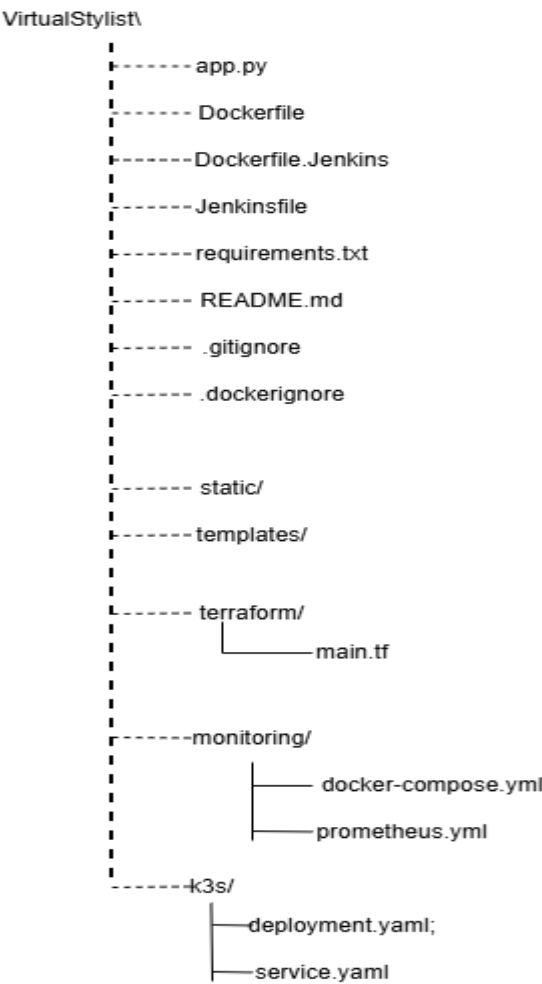
Frontend technologies used:

- HTML
- CSS

- JavaScript

The frontend was designed with a modern and responsive interface for improved user experience.

Project Structure



Infrastructure Setup on AWS

AWS EC2 was used to host the complete DevOps environment.

EC2 Instance Configuration

Configuration	Value
Cloud Provider	AWS
Service Used	EC2
Operating System	Ubuntu

Instance Type	t3.small
Region	ap-south-1

Security Group Configuration

Inbound rules were configured to allow access to:

Port	Purpose
22	SSH
5000	Flask Application
8080	Jenkins
3000	Grafana
9090	Prometheus
30007	Kubernetes NodePort

Docker Containerization

The application was containerized using Docker.

Dockerfile Responsibilities

The Dockerfile performed the following tasks:

- Selected Python base image
- Installed dependencies
- Copied source code
- Exposed required port
- Started Flask application

Docker Build Command

`docker build -t virtualstylist .`

Docker Run Command

```
docker run -p 5000:5000 virtualstylist
```

Benefits Achieved

- Portability
- Isolation
- Consistency across environments
- Simplified deployment

Jenkins CI/CD Pipeline Setup

Jenkins was deployed inside a Docker container.

Jenkins Configuration

Jenkins was configured with:

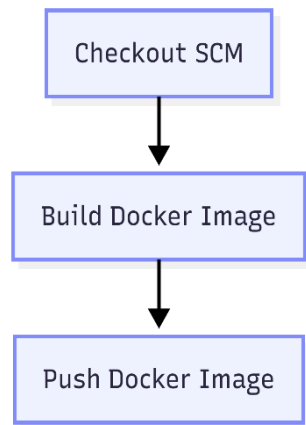
- Docker socket mounting
- DockerHub credentials
- GitHub repository integration
- Pipeline configuration

Pipeline Workflow

The Jenkins pipeline automatically:

1. Pulls latest code from GitHub
2. Builds Docker image
3. Pushes image to DockerHub

Jenkinsfile Example Workflow



Benefits of CI/CD

- Faster deployments
- Reduced manual work
- Continuous integration
- Automated build management

DockerHub Integration

DockerHub was used as a container registry.

The Docker image generated by Jenkins was pushed to DockerHub for centralized image storage.

DockerHub Benefits

- Image versioning
- Remote storage
- Easy deployment
- Scalability

Kubernetes Deployment using K3s

K3s, a lightweight Kubernetes distribution, was used for container orchestration.

Kubernetes Components Used

Deployment

Responsible for:

- Pod management
- Replica maintenance
- Automatic restart
- Rolling updates

Service

Responsible for:

- External access
- NodePort exposure
- Load balancing

Kubernetes Commands Used

```
kubectl apply -f deployment.yaml
```

```
kubectl apply -f service.yaml
```

```
kubectl get pods
```

```
kubectl get svc
```

Kubernetes Features Achieved

- Self-healing
- Scalability
- Container orchestration
- Simplified management

Monitoring Stack Implementation

A monitoring stack was implemented using Docker Compose.

Components

Component	Function
Node Exporter	Collect metrics

Prometheus	Store metrics
Grafana	Visualize metrics

Docker Compose Services

The monitoring stack included:

- Prometheus container
- Grafana container
- Node Exporter container

Metrics Monitored

- CPU utilization
- Memory usage
- Disk usage
- Network traffic
- System uptime

Grafana Dashboard Setup

Grafana dashboards were configured using Prometheus as the data source.

Dashboard Features

- Real-time monitoring
- Interactive graphs
- System visualization
- Performance tracking

Dashboard Metrics

- CPU usage graph

- Memory utilization graph
- Network traffic graph
- Disk space graph

Final Deployment

The application was successfully deployed on AWS EC2 using Kubernetes.

The deployed application was accessible publicly through the Kubernetes NodePort service.

The final deployment demonstrated:

- Automated CI/CD pipeline
- Containerized deployment
- Cloud hosting
- Kubernetes orchestration
- Real-time monitoring

Terraform Configuration

Terraform was used to provision AWS infrastructure automatically.

Features

- Infrastructure as Code
- Automated EC2 provisioning
- Security group creation
- Scalable infrastructure management

Benefits

- Reduced manual configuration
- Reusable infrastructure templates
- Improved consistency
- Faster provisioning

Terraform Workflow

terraform init

terraform plan

terraform apply

Docker Containerization

Docker was used to package the application along with all dependencies.

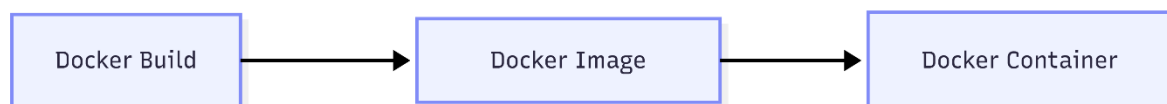
Dockerfile Responsibilities

- Install dependencies
- Copy application code
- Configure runtime environment
- Expose application ports
- Start Flask application

Advantages of Docker

- Environment consistency
- Lightweight deployment
- Portability
- Fast startup
- Dependency isolation

Docker Workflow



Jenkins CI/CD Pipeline

Jenkins was used to automate software delivery.

Pipeline Stages

1. Clone Repository
2. Build Docker Image
3. Push Docker Image to DockerHub

Advantages

- Automated deployment
- Reduced manual effort
- Faster software delivery
- Continuous integration
- Continuous deployment

Jenkinsfile

The Jenkins pipeline was implemented using a declarative Jenkinsfile.

Kubernetes Deployment

Kubernetes (K3s) was used for container orchestration.

Kubernetes Components Used

Deployment

Manages application pods.

Service

Exposes application to external users.

Pod

Runs application container.

Advantages of Kubernetes

- High availability
- Scalability
- Self-healing
- Load balancing

- Rolling updates

Deployment Process

```
kubectl apply -f deployment.yaml
```

```
kubectl apply -f service.yaml
```

Monitoring using Prometheus and Grafana

Monitoring was implemented using Prometheus and Grafana.

Node Exporter

Node Exporter collects system-level metrics such as:

- CPU usage
- Memory usage
- Disk usage
- Network statistics

Prometheus

Prometheus periodically scrapes metrics from Node Exporter.

Grafana

Grafana visualizes metrics using dashboards.

Monitoring Features

- Real-time visualization
- Performance tracking
- Resource monitoring
- System health analysis

Testing and Validation

The deployed application was tested on AWS EC2.

Tests Performed

Application Testing

Verified web application functionality.

Docker Testing

Verified container build and execution.

Jenkins Testing

Verified CI/CD pipeline execution.

Kubernetes Testing

Verified pod deployment and service accessibility.

Monitoring Testing

Verified Prometheus target health and Grafana dashboards.

Test Results

All major components functioned successfully.

Advantages of the System

The proposed system offers several advantages:

- Automated deployment
- Reduced deployment time
- Improved scalability
- Better monitoring
- High reliability
- Infrastructure automation
- Faster software delivery
- Improved consistency
- Simplified management

Challenges Faced

Several challenges were encountered during implementation:

- Memory limitations on cloud instance

- Kubernetes configuration issues
- Jenkins Docker permission problems
- Docker socket access issues
- CI/CD pipeline failures during setup
- Monitoring configuration setup

These issues were resolved through troubleshooting and optimization.

Future Enhancements

Possible future enhancements include:

- AI-powered recommendation engine
- Auto-scaling Kubernetes clusters
- Webhook automation
- HTTPS integration
- Multi-cloud deployment
- Advanced security implementation
- Machine learning integration
- Mobile application support
- Load balancing enhancements

Conclusion

This project successfully demonstrated the deployment of a Virtual Stylist application using modern DevOps and cloud technologies.

The implementation integrated multiple tools including Terraform, Docker, Jenkins, Kubernetes, AWS EC2, Prometheus, and Grafana to create a fully automated deployment and monitoring pipeline.

The project achieved its primary objectives of automation, scalability, monitoring, and continuous deployment. The implementation reduced manual deployment efforts while improving consistency and system reliability.

The project also provided practical exposure to industry-standard DevOps practices and cloud-native application management.

Overall, the project demonstrates how DevOps tools and cloud technologies can be effectively integrated to build scalable and automated software deployment systems.

References

1. Docker Documentation <https://docs.docker.com/>
2. Kubernetes Documentation – <https://kubernetes.io/docs/>
3. Jenkins Documentation – <https://www.jenkins.io/doc/>
4. Terraform Documentation – <https://developer.hashicorp.com/terraform/docs>
5. AWS EC2 Documentation – <https://docs.aws.amazon.com/ec2/>
6. Prometheus Documentation – <https://prometheus.io/docs/>
7. Grafana Documentation – <https://grafana.com/docs/>
8. Flask Documentation – <https://flask.palletsprojects.com/>
9. GitHub Documentation – <https://docs.github.com/>
10. DockerHub Documentation – <https://docs.docker.com/docker-hub/>

Appendix

Appendix A: Screenshots

```
(venv) PS D:\VirtualStylist\terraform> terraform apply
aws_security_group.virtualstylist_sg: Refreshing state... [id=sg-054c7d17ef5345a53]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with
the following symbols:
  + create

Terraform will perform the following actions:

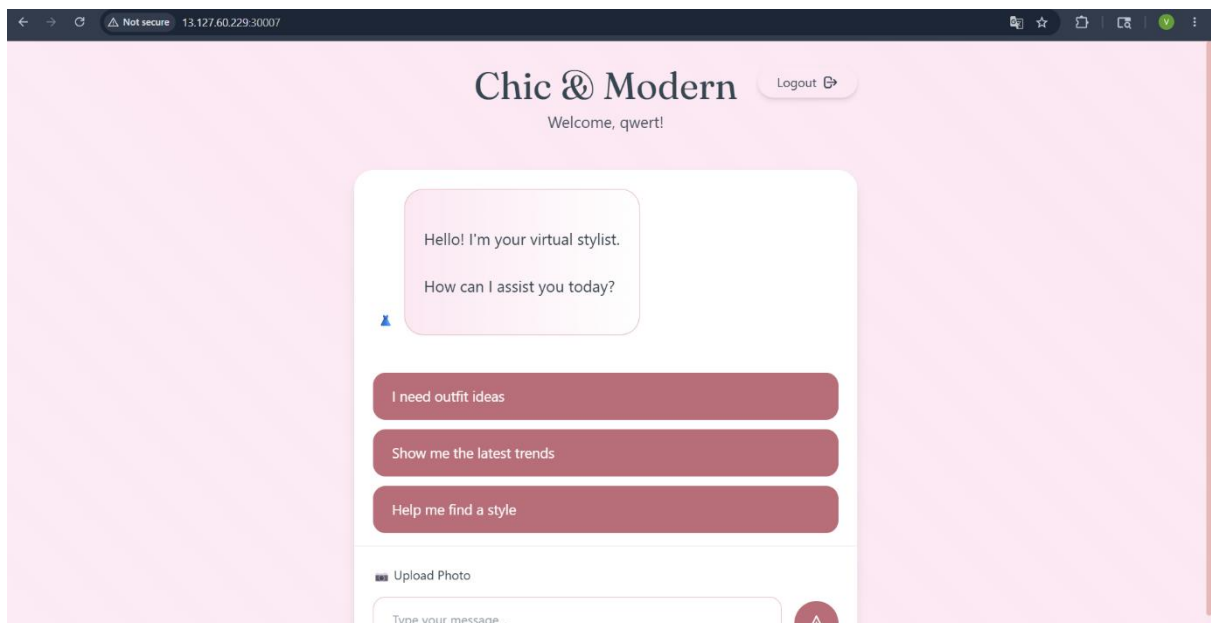
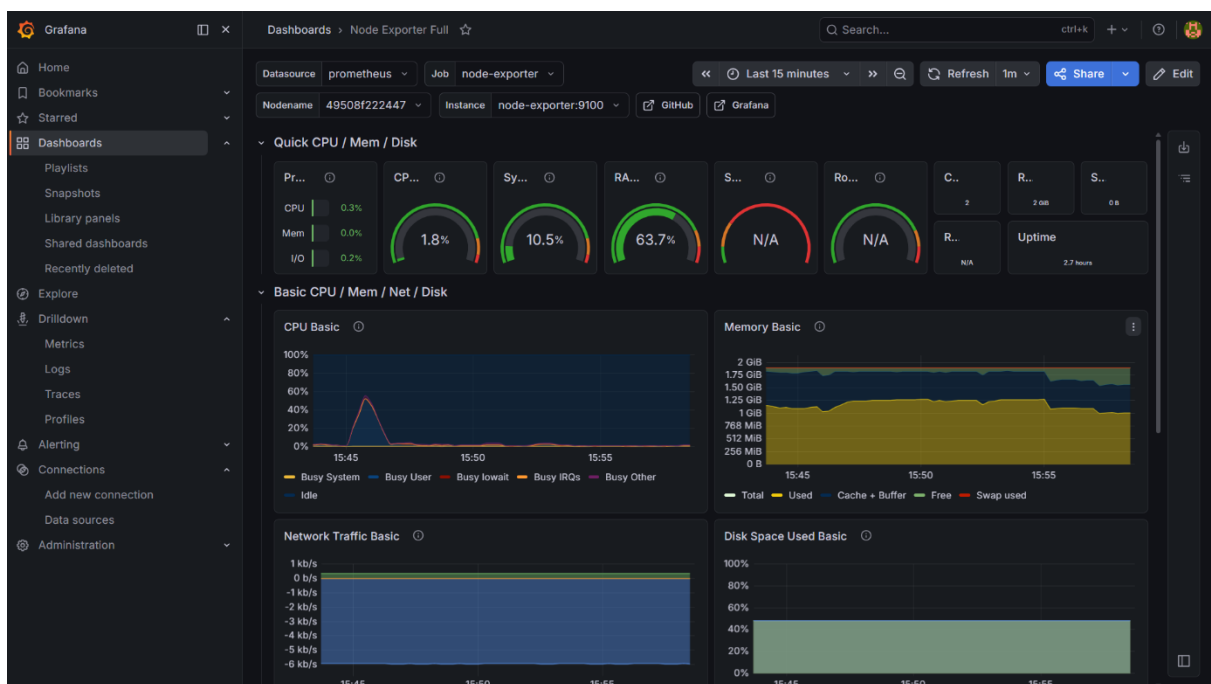
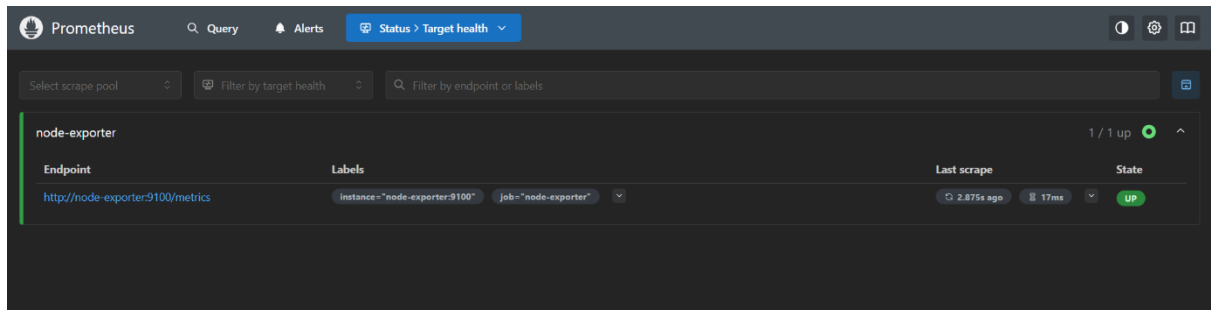
# aws_instance.virtualstylist_server will be created
+ resource "aws_instance" "virtualstylist_server" {
  + ami                  = "ami-0f5ee92e2d63afc18"
  + arn                  = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone     = (known after apply)
  + disable_api_stop      = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized         = (known after apply)
  + enable_primary_ipv6   = (known after apply)
  + force_destroy         = false
  + get_password_data     = false
  + host_id               = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile   = (known after apply)
  + id                    = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle    = (known after apply)
  + instance_state        = (known after apply)
  + instance_type         = "t3.micro"
  + ipv6_address_count    = (known after apply)
  + ipv6_addresses        = (known after apply)
  + key_name              = "virtualstylist-key"
  + monitoring            = (known after apply)
  + outpost_arn           = (known after apply)
  + password_data         = (known after apply)
  + placement_group       = (known after apply)
```



```

ubuntu@ip-172-31-12-63:~$ sudo k3s kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
virtualstylist-deployment-7b988b57d9-spfxb  1/1     Running   1 (5m44s ago)  28m
ubuntu@ip-172-31-12-63:~$ sudo k3s kubectl get svc
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes          ClusterIP   10.43.0.1    <none>        443/TCP          32m
virtualstylist-service NodePort    10.43.107.70 <none>        5000:30007/TCP  18s

```



Appendix B: Commands Used

Docker Commands

`docker build -t virtualstylist .`

`docker run -p 5000:5000 virtualstylist`

Kubernetes Commands

`kubectl apply -f deployment.yaml`

`kubectl apply -f service.yaml`

`kubectl get pods`

`kubectl get svc`

Terraform Commands

`terraform init`

`terraform plan`

`terraform apply`

Jenkins Pipeline Commands

`docker build -t image-name .`

`docker push image-name`